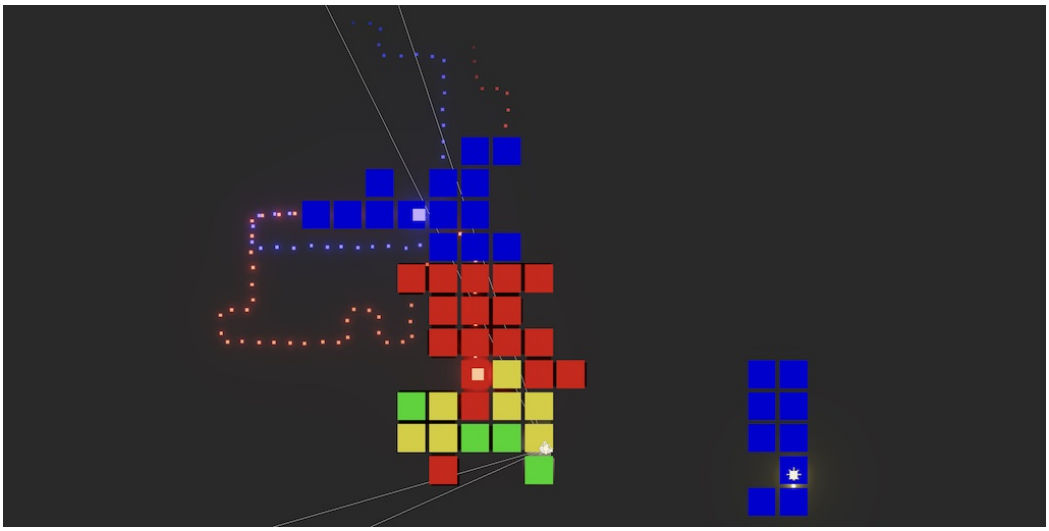
published 2023-09-25

Maze 2.1.0 Occlusion Culling

Scan cells to determine visibility.
Detect overlapping vision of agents and player.
Deactivate hidden cells and lights.



Determining which cells to render, skipping most of the maze.

This tutorial is made with Unity 2022.3.9f1 and follows Maze 2.

1 Vision

In this tutorial we'll add occlusion culling to the maze, so that we can avoid rendering cells that are not visible. This is beneficial besides frustum culling and depth sorting because it reduces the amount of draw calls and the amount of geometry that needs to be depth-tested. It will also allow us to disable shadow-casting lights that are fully occluded.

We're going to use a technique inspired by the FOV (field of view) using recursive shadowcasting algorithm by Björn Bergström, but iterative, with blocking edges instead of cells, and running on quadrants instead of octants.

We start by determining visibility in the northeastern quadrant relative to the player only, then later generalize this approach to work for all directions.

1.1 Tracking Visibility

To track the visibility of a cell we add a bit flag for it to `MazeFlags`.

```
[System.Flags]
public enum MazeFlags
{
	…

	PassagesDiagonal = 0b1111_0000,

	Visible = 0b1_0000_0000
}
```

The first step of the algorithm is to clear visibility, for which we can use a parallel `ClearOcclusionJob` job.

```
using Unity.Burst;
using Unity.Jobs;

[BurstCompile]
public struct ClearOcclusionJob : IJobFor
{
	public Maze maze;

	public void Execute(int i) => maze.Unset(i, MazeFlags.Visible);
}
```

The rest of the algorithm will run in a separate `OcclusionJob` job that will process a single quadrant. It is also a parallel job because we can process the quadrants in parallel. Besides the maze it needs to know the view position and how much the walls extent into the cells. Make it do nothing for now. We'll use a few struct types to segregate the code of this job, which we'll put in different script assets, so define it as a partial struct.

```
using Unity.Burst;
using Unity.Collections;
using Unity.Jobs;
using Unity.Mathematics;

using static Unity.Mathematics.math;

[BurstCompile]
public partial struct OcclusionJob : IJobFor
{
	public Maze maze;

	public float2 position;

	public float wallExtents;

	public void Execute(int i) { }
}
```

The position and wall extents are in maze space. Add methods to `Maze` to convert from world to maze distance and from world to maze position.

```
	public readonly float WorldToMazeDistance(float distance) => distance * 0.5f;

	public readonly float2 WorldToMazePosition(Vector3 position) =>
		(float2(position.x, position.z) + size) * 0.5f;
```

Add a configuration field for the wall extents to `Game`. We designed our walls to extent 0.25 world units into the cells so we use that as the default. You could modify this value at runtime to see how wall thickness affects vision.

Schedule and run the jobs in a new `UpdateOcclusion` method and invoke it at the end of `UpdateGame`, passing it the player position. Because clearing the visibility data is so little work Burst partially unrolls the loop. So trying to schedule it on many threads is counterproductive. Let's use a quarter of the maze length as the batch size. Also, because we'll only process the northeastern quadrant for now the main occlusion job only needs a single iteration, though we'll increase this to four later.

```
	[SerializeField, Range(0, 0.5f)]
	float wallExtents = 0.25f;

	…

	void UpdateGame()
	{
		…
		UpdateOcclusion(playerPosition);
	}

	void UpdateOcclusion(Vector3 playerPosition)
	{
		JobHandle handle = new ClearOcclusionJob
		{
			maze = maze
		}.ScheduleParallel(maze.Length, maze.Length / 4, default);

		handle = new OcclusionJob
		{
			maze = maze,
			position = maze.WorldToMazePosition(playerPosition),
			wallExtents = maze.WorldToMazeDistance(wallExtents)
		}.ScheduleParallel(1, 1, handle);

		handle.Complete();
	}
```

1.2 Row Scans

Our algorithm works by scanning a row of cells, going eastward. When certain conditions are met it'll schedule a scan of another row, north of the current one. Each scan has a starting point, for which we need a cell index and an offset vector. The vision cone used for the scan can be defined by two lines, the left and the right one from the point of view of the player. We only need to store the slopes of these lines. Create an inner `Scan` struct type for this, put in a different asset named *OcclusionJob.Scan*.

```
using Unity.Mathematics;

using static Unity.Mathematics.math;

partial struct OcclusionJob
{
	struct Scan
	{
		public int index;

		public float2 offset;

		public float leftSlope, rightSlope;
}
```

While scanning a row we'll encounter cells where a new scan should start. We can accommodate this by shifting the existing scan eastward to the current cell. Add a `ShiftEast` method for this, which replaces the index, X offset, and left slope.

```
		public void ShiftEast(int index, float xOffset, float leftSlope)
		{
			this.index = index;
			offset.x = xOffset;
			this.leftSlope = leftSlope;
		}
```

Also add a `ShiftedNorth` method that returns a new scan by shifting the current one north, adding an index offset, incrementing the Y offset, and providing a new right slope.

```
		public readonly Scan ShiftedNorth(int indexOffset, float rightSlope) => new()
		{
			index = index + indexOffset,
			offset = float2(offset.x, offset.y + 1f),
			leftSlope = leftSlope,
			rightSlope = rightSlope
		};
	}
```

To avoid recursion we'll store these scans in a stack. We don't need a generic dynamically-resizing stack as its maximum size is known. So let's create a simple inner `ScanStack` struct type with a constructor to create it with a capacity and first scan, a method to push a scan, and a method that tries to pop a scan.

```
using Unity.Collections;

partial struct OcclusionJob
{
	struct ScanStack
	{
		NativeArray<Scan> stack;

		int stackSize;

		public ScanStack(int capacity, Scan firstScan)
		{
			stack = new(capacity, Allocator.Temp, NativeArrayOptions.UninitializedMemory);
			stack[0] = firstScan;
			stackSize = 1;
		}

		public void Push(Scan scan) => stack[stackSize++] = scan;

		public bool TryPop(out Scan scan)
		{
			if (stackSize > 0)
			{
				scan = stack[--stackSize];
				return true;
			}
			scan = default;
			return false;
		}
	}
}
```

1.3 Cell Data

We need to analyze each cell that gets scanned. Let's put all code for that in an inner `CellData` struct type to keep it isolated. We need to know the inset used for walls and the single-dimensional offsets for the south, north, west, and east edges of the cell. We'll initialize and use one data value per scan, so its south and north offsets are constant, as well as the inset.

```
using Unity.Mathematics;

using static Unity.Mathematics.math;

partial struct OcclusionJob
{
	struct CellData
	{
		readonly float inset;

		readonly float south, north;

		float west, east;
	}
}
```

The questions that we're trying to answer is whether the north and the east neighbors of this cell are visible and what the left and right slopes of the vision cone for this cell are. Add properties for these that are set privately.

```
		public bool IsNorthVisible
		{ get; private set; }

		public bool IsEastVisible
		{ get; private set; }

		public float LeftSlope
		{ get; private set; }

		public float RightSlope
		{ get; private set; }
```

Its constructor method takes an offset vector, inset, and initial slopes as input and uses that to initialize the data. The offset vector points to the southwest corner of the cell. Also give it a `StepEast` method which increments both the west and east offsets.

```
		public CellData(float2 offset, float inset, float leftSlope, float rightSlope)
		{
			this = default;
			this.inset = inset;
			west = offset.x;
			east = left + 1f;
			south = offset.y;
			north = south + 1f;
			LeftSlope = leftSlope;
			RightSlope = rightSlope;
		}

		public void StepEast()
		{
			west += 1f;
			east += 1f;
		}
```

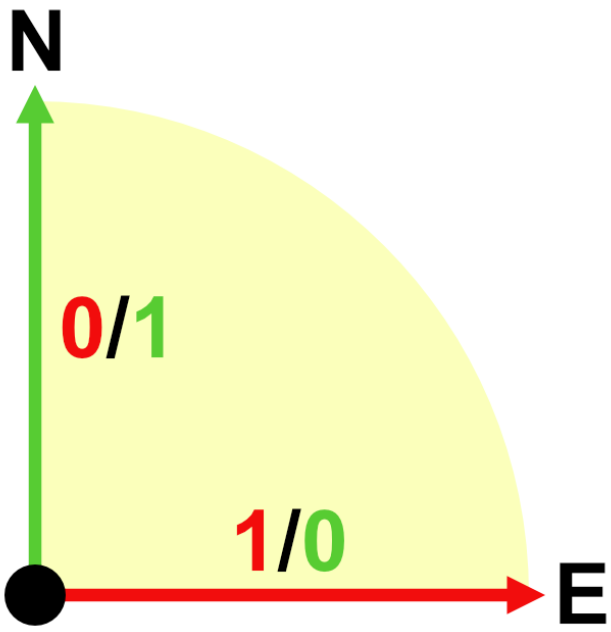
The cell evaluation will happen in an `UpdateForNextCell` method, which will use the flags of a given cell to set its properties. Leave it empty for now.

```
		public void UpdateForNextCell(MazeFlags cell) { }
```

1.4 Scanning

Now that we have the helper structs we can begin to implementing `OcclusionJob.Execute`. The first step is to get the first scan, along with the origin of the vision cone. If it possible that we end up without a first scan, in which case the job can abort, so we put that code in a `TryGetFirstScan` method with the scan and origin as output parameters.

At this point we're not yet going to avoid scanning so always have `TryGetFirstScan` indicate success. The scan origin is the player's position in the starting cell, which is the fractional part of its maze position. The scan index is found by converting the player's integer coordinates to a cell index. As we're evaluating the entire northeast quadrant the left line of the vision cone points north and the right line points east. As we're scanning eastward let's find the slopes by dividing the east offset by the north offset so the slope increases the more it points eastward. Thus the left slope is zero and the right slope would approach infinity, but let's use `float.MaxValue` for that.



Northeast quadrant vision cone.

```
public void Execute(int i)
{
    if (!TryGetFirstScan(out Scan scan, out float2 origin))
    {
        return;
    }
}

bool TryGetFirstScan(out Scan scan, out float2 origin)
{
    origin = frac(position);
    scan = new Scan
    {
        index = maze.CoordinatesToIndex((int2)position),
        rightSlope = float.MaxValue
    };
    return true;
}
```

Our algorithm then creates the stack and while it can pop a scan will create new cell data, passing it the relative scan offset, wall extents, and the scan's slopes. It should also keep track of the current index and X offset. The stack's capacity should be equal to the east-west size of the maze, as that's the worst-case amount of scans that we'll have to track, pushing a new one per cell in a row.

```
	public void Execute(int i)
	{
		…

		var stack = new ScanStack(maze.SizeEW, scan);
		while (stack.TryPop(out scan))
		{
			var data = new CellData(
				scan.offset - origin, wallExtents, scan.leftSlope, scan.rightSlope);
			int currentIndex = scan.index;
			float currentXOffset = scan.offset.x;
		}
	}
```

For each scan we enter a loop to to worth through the row. Each iteration we update the data for the current cell, which we also mark as visible. Then if the east neighbor is visible we increase the index and X offset and step to the east and continue, otherwise we stop.

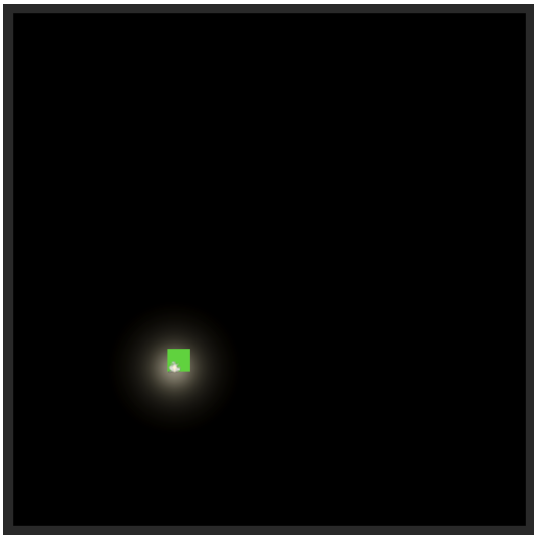
```
			float currentXOffset = scan.offset.x;

			while (true)
			{
				MazeFlags cell = maze.Set(currentIndex, MazeFlags.Visible);
				data.UpdateForNextCell(cell);

				if (data.IsEastVisible)
				{
					currentIndex += maze.StepE;
					currentXOffset += 1f;
					data.StepEast();
				}
				else
				{
					break;
				}
			}
```

1.5 Debug Visualization

Because we don't update the cell data yet our algorithm will stop after the first cell. So it marks the cell the player occupies as visible and nothing else. Let's take this moment to add a debug visualization of the cell visibility so we can inspect the results of our algorithm. We do this by adding an `OnDrawGizmosSelected` method to `Game` which, if it is playing, loops through all cells and draws a green tile on the bottom of each visible cell. Make theses tiles 1.75 units wide so that we can see the edges between visible cells.

```
	void OnDrawGizmosSelected()
	{
		if (!isPlaying)
		{
			return;
		}

		Gizmos.color = Color.green;
		var size = new Vector3(1.75f, 0.01f, 1.75f);
		for (int i = 0; i < cellObjects.Length; i++)
		{
			if (maze[i].Has(MazeFlags.Visible))
			{
				Vector3 position = cellObjects[i].transform.localPosition;
				position.y = 0f;
				Gizmos.DrawCube(position, size);
			}
		}
	}
```

Now we'll see the tile gizmos in the scene window while we have *Game* selected. Gizmos can also be enabled in the game window to get a first-person view of them. To test whether the algorithm will process an entire quadrant set the *Open Arbitrary Probability* of *Game* to 1.



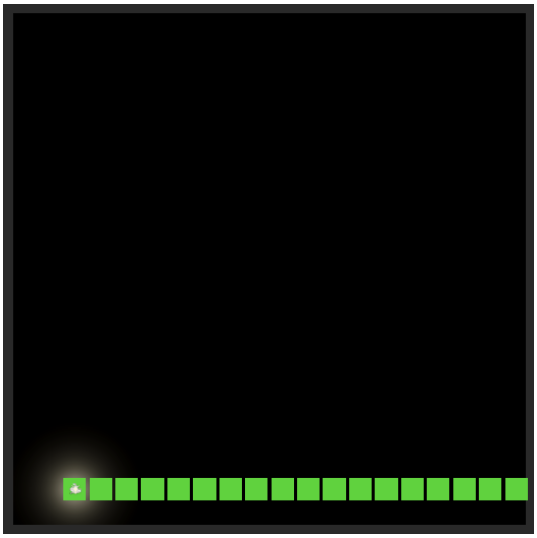
Gizmos showing one visible cell.

Temporarily clear the *Agents* array so they don't interrupt our work by catching the player.

1.6 Scanning a Quadrant

Going back to our algorithm, to make it scan more cells we have to indicate whether north and east are visible in `CellData.UpdateForNextCell` based on the cell flags.

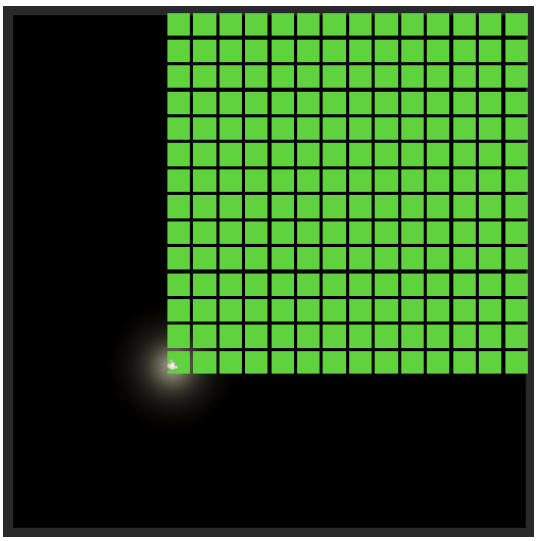
```
	public void UpdateForNextCell(MazeFlags cell) {
		IsNorthVisible = cell.Has(MazeFlags.PassageN);
		IsEastVisible = cell.Has(MazeFlags.PassageE);
	}
```



Visible row; from player to eastern edge.

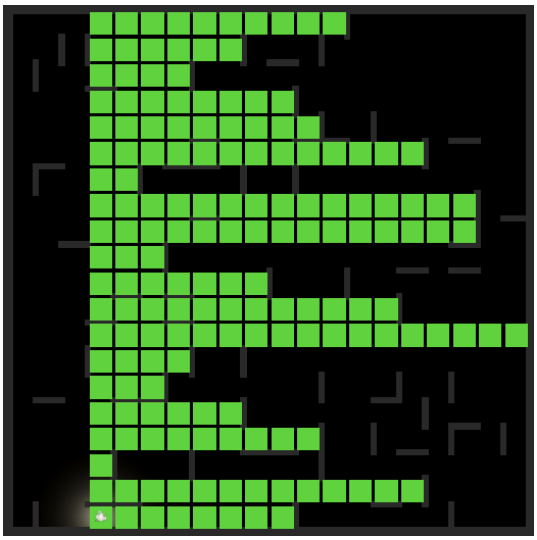
Now we scan the entire row from the player to the eastern edge of the empty maze. To also go north, when we can no longer go east, shift the current scan north and push it onto the stack before stopping, in `OcclusionJob.Execute`.

```
			if (data.IsEastVisible)
			{
				currentIndex += maze.StepE;
				currentXOffset += 1f;
				data.StepEast();
			}
			else
			{
				if (data.IsNorthVisible)
				{
					stack.Push(scan.ShiftedNorth(maze.StepN, data.RightSlope));
				}
				break;
			}
```



Visible quadrant; from player to north and east maze edges.

Let's see how our algorithm behaves when we add walls. Set *Seed* to 123 and *Open Arbitrary Probability* to 0.75.



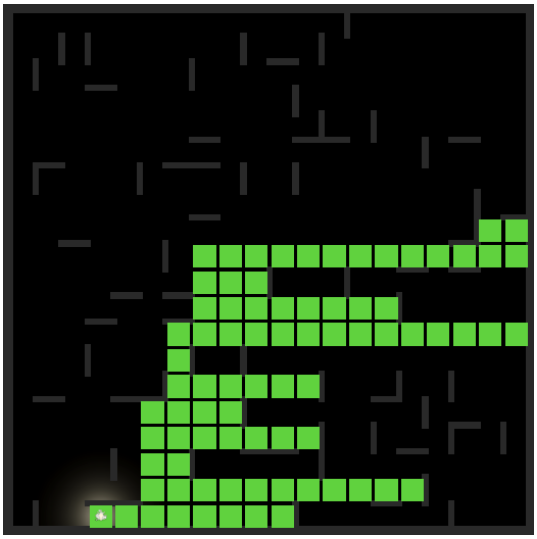
Walls block row scans.

We're scanning eastward until hitting a wall, then start a new scan one step north from the start of the current scan.

1.7 Shifting Scans

To also take northern visibility into account we have to keep track of whether north is visible from the previous cell, which is initially not the case as it is out of view. Then while scanning first check whether north is visible before checking east. If north is visible and it previously wasn't, then we've found the first cell for a next scan and shift east to match. Then, if we can go east, update the previous northern visibility accordingly.

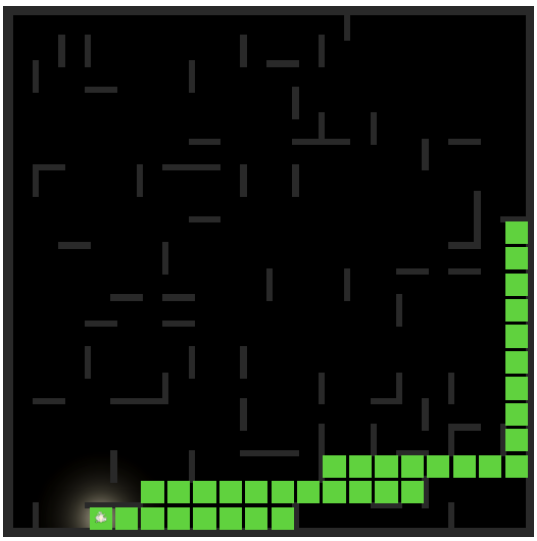
```
		bool isPreviousNorthVisible = false;

		while (true)
		{
			MazeFlags cell = maze.Set(currentIndex, MazeFlags.Visible);
			data.UpdateForNextCell(cell);

			if (data.IsNorthVisible)
			{
				if (!isPreviousNorthVisible)
				{
					scan.ShiftEast(currentIndex, currentXOffset, data.LeftSlope);
				}
			}

			if (data.IsEastVisible)
			{
				currentIndex += maze.StepE;
				currentXOffset += 1f;
				isPreviousNorthVisible = data.IsNorthVisible;
				data.StepEast();
			}
			else
			{ … }
		}
```



Shifted scans.

This shifts scan eastward so they begin after the last cell that had a northern wall. But a new scan should also start if the northern cell has a western wall, as that also blocks vision. This is the case if there isn't a northwest passage while north is open. So in that case also shift right.

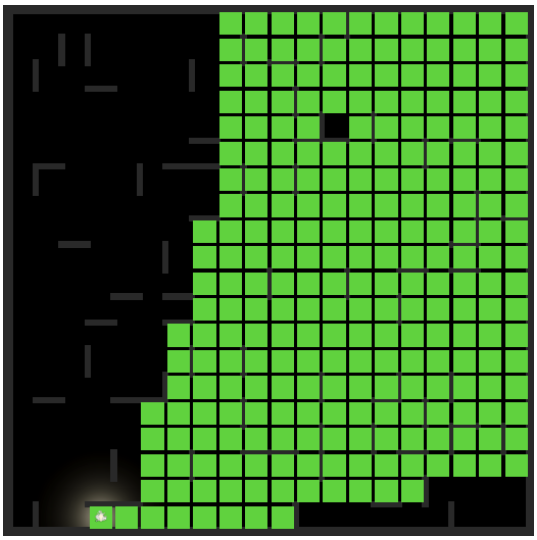
```
				if (!isPreviousNorthVisible)
				{
					scan.ShiftEast(currentIndex, currentXOffset, data.LeftSlope);
				}
				else if (cell.HasNot(MazeFlags.PassageNW))
				{
					scan.ShiftEast(currentIndex, currentXOffset, data.LeftSlope);
				}
```



Shifting past visual obstructions.

At this point we add scans only for the last open segment of each row. We also have to add scans when we detect the end of earlier open segments. This is the case when current north is closed while previous north is open, and also when we encounter a western wall north of us.

```
if (data.IsNorthVisible)
{
    if (!isPreviousNorthVisible)
    {
        scan.ShiftEast(currentIndex, currentXOffset, data.LeftSlope);
    }
    else if (cell.HasNot(MazeFlags.PassageNW))
    {
        stack.Push(scan.ShiftedNorth(maze.StepN, data.RightSlope));
        scan.ShiftEast(currentIndex, currentXOffset, data.LeftSlope);
    }
}
else if (isPreviousNorthVisible)
{
    stack.Push(scan.ShiftedNorth(maze.StepN, data.RightSlope));
}
```



Maximum scans going north and east.

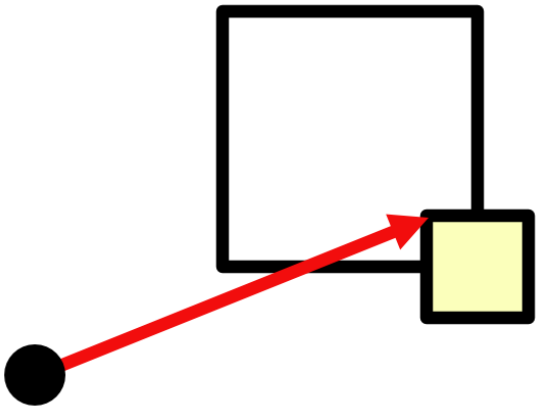
1.8 Constraining Vision

By now our algorithm marks all cells as visible that could be reached by going east and north only. To narrow this down further we have to take the vision cone into consideration. The idea is that each cell that we visit potentially constrains what we can see further away. We implement that by increasing the left slope and decreasing the right slope when needed. Adjusting these slopes is the responsibility of `CellData`.

If our walls didn't have any thickness only checking corners would suffice, but because walls do have thickness we'll also have to check cell edges adjusted by the wall inset. Add convenient inset getter properties for the four edges.

```
		readonly float WestInset => west + inset;

		readonly float EastInset => east - inset;

		readonly float SouthInset => south + inset;

		readonly float NorthInset => north - inset;
```

The first thing that we'll do in `UpdateForNextCell` is constrain the right slope. This could be the case when there is a southern passage but not a southeastern one, because the southeast neighbor has a west or north wall that pokes into this cell. The right slope then becomes the slope for the inset southeastern corner, if that is less than our current right slope. But we should only do this when the south inset is positive, otherwise it lies to the south of the quadrant, and a division would make the slope either infinite or negative.



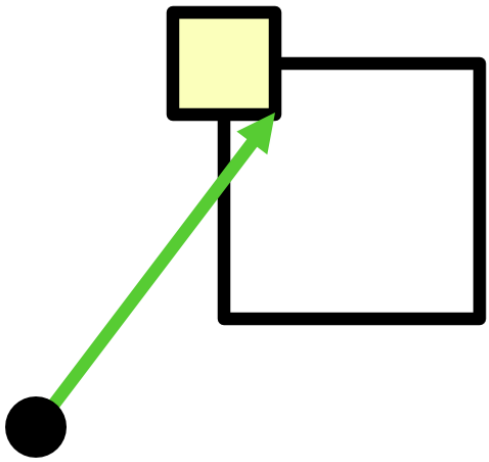
Constrained by inset southeast corner.

```
        public void UpdateForNextCell(MazeFlags cell, Quadrant quadrant, float range) {
            if (cell.Has(MazeFlags.PassageS) &&
                cell.HasNot(MazeFlags.PassageSE) &&
                SouthInset > 0f)
            {
                RightSlope = min(RightSlope, EastInset / SouthInset);
            }

            …
        }
```

What if there is a south wall?

Then the right slope would've already been constrained earlier, if needed.

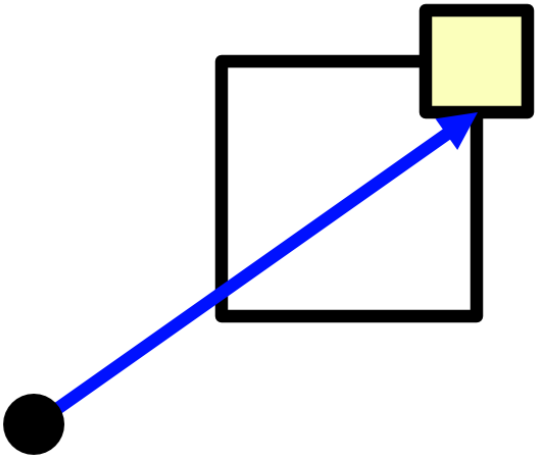
We can do the same for the left slope and the northwest corner, taking the maximum of that slope and the current left slope. We can do this while checking whether north is open. Also, after potentially updating the left slope, north can only be visible if the left slope is less than the right slope, otherwise vision is completely blocked.



Constrained by inset northwest corner.

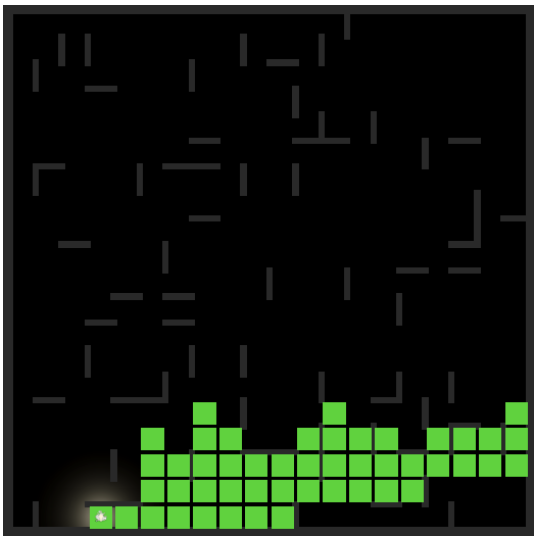
```
            //IsNorthVisible = cell.Has(MazeFlags.PassageN);
            if (cell.Has(MazeFlags.PassageN))
            {
                if (cell.HasNot(MazeFlags.PassageNW) && WestInset > 0f)
                {
                    LeftSlope = max(LeftSlope, WestInset / NorthInset);
                }
                IsNorthVisible = LeftSlope < RightSlope;
            }
            else
            {
                IsNorthVisible = false;
            }
```

Also refine the check whether east is open, adding the check for complete view blockage. Furthermore, let's consider a cell with an open northeast corner. If that corner lies south of the right slope then the east cell is fully out of view. If that corner is blocked instead we check along the cell east edge with north inset. But only if the inset lies inside the view quadrant, otherwise the corner blocks our entire view eastward.



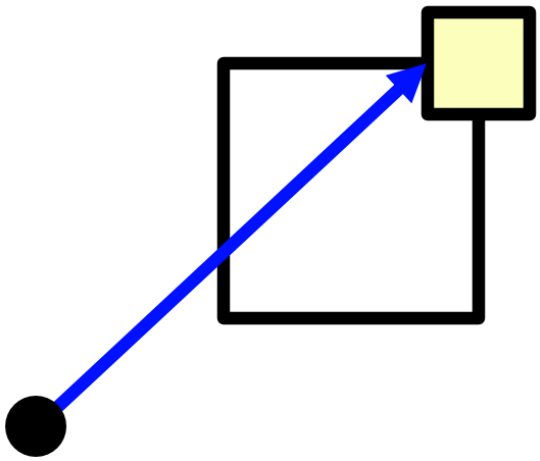
East visibility check with blocked corner.

```
		IsEastVisible =
			cell.Has(MazeFlags.PassageE) &&
			LeftSlope < RightSlope &&
			(cell.Has(MazeFlags.PassageNE) ?
				east / north < RightSlope :
				NorthInset > 0f && east / NorthInset < RightSlope);
```



Constraining vision.

Perform a similar check with the left slope and the northeast corner for northern visibility. In this case we don't care if the inset ends up outside the view quadrant because a negative left slope is fine and we won't get a division by zero. The slope that we test here will also be needed for northern scans that we might push later, if we take the minimum of it and the current right slope. So let's expose it via a `RightSlopeForNorthNeighbor` property. We then also only have to compare the left slope with this one.

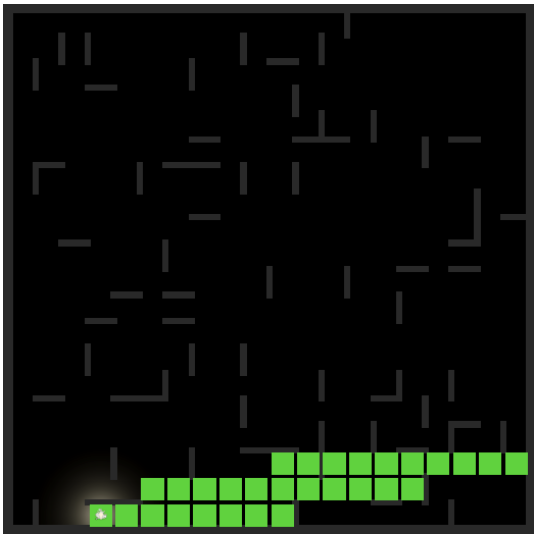


North visibility check with blocked corner.

```
	public float RightSlopeForNorthNeighbor
	{ get; private set; }

	…

	public void UpdateForNextCell(MazeFlags cell) {
		…

		if (cell.Has(MazeFlags.PassageN))
		{
			…

			RightSlopeForNorthNeighbor = min(RightSlope,
				(cell.Has(MazeFlags.PassageNE) ? east : EastInset) / north);

			IsNorthVisible = LeftSlope < RightSlopeForNorthNeighbor;
		}
		else
		{
			IsNorthVisible = false;
		}

		…
	}
```

Each time we push a new scan in `OcclusionJob.Execute` we can use that slope for the right slope. However, except when encountering an east wall, the shift north happens based on the previously visited cell, so we have to remember the slope of the previous iteration.

```
		float previousRightSlopeForNorthNeighbor = 0f;

		while (true)
		{
			MazeFlags cell = maze.Set(currentIndex, MazeFlags.Visible);
			data.UpdateForNextCell(cell);

			if (data.IsNorthVisible)
			{
				if (!isPreviousNorthVisible)
				{
					scan.ShiftEast(currentIndex, currentXOffset, data.LeftSlope);
				}
				else if (cell.HasNot(MazeFlags.PassageNW))
				{
					stack.Push(scan.ShiftedNorth(
						maze.StepN, previousRightSlopeForNorthNeighbor));
					scan.ShiftEast(currentIndex, currentXOffset, data.LeftSlope);
				}
			}
			else if (isPreviousNorthVisible)
			{
				stack.Push(scan.ShiftedNorth(
					maze.StepN, previousRightSlopeForNorthNeighbor));
			}

			if (data.IsEastVisible)
			{
				currentIndex += maze.StepE;
				currentXOffset += 1f;
				isPreviousNorthVisible = data.IsNorthVisible;
				previousRightSlopeForNorthNeighbor = data.RightSlopeForNorthNeighbor;
				data.StepEast();
			}
			else
			{
				if (data.IsNorthVisible)
				{
					stack.Push(scan.ShiftedNorth(
						maze.StepN, data.RightSlopeForNorthNeighbor));
				}
				break;
			}
		}
```



Fully constrained vision.

2 Quadrants

Our occlusion algorithm works, but currently only for everything in the northeast quadrant relative to the player. To support full omnidirectional vision we'll run the algorithm for the other three quadrants as well. This means that the quadrants will overlap at least partially along the first row or column. That is fine because we're only enabling visibility, so which quadrant updates a cell last doesn't matter.

2.1 Relative Directions

Our algorithm doesn't care in which direction it runs, as long as it is consistent. We'll support the other quadrants by flipping north-south, east-west, or both dimensions. To do this we introduce an inner `Quadrant` struct type that contains the flip state and flags for all passages that we need to check, which are north, east, south, northwest, northeast, and southeast. Give it a constructor that sets these flags based on given flip states.

```
partial struct OcclusionJob
{
    readonly struct Quadrant
    {
        public readonly MazeFlags north, east, south, northwest, northeast, southeast;

        public readonly bool flipNS, flipEW;

        public Quadrant(bool flipNS, bool flipEW)
        {
            this.flipNS = flipNS;
            this.flipEW = flipEW;

            north = flipNS ? MazeFlags.PassageS : MazeFlags.PassageN;
            south = flipNS ? MazeFlags.PassageN : MazeFlags.PassageS;
            east = flipEW ? MazeFlags.PassageW : MazeFlags.PassageE;

            if (flipEW)
            {
                if (flipNS)
                {
                    northwest = MazeFlags.PassageSE;
                    northeast = MazeFlags.PassageSW;
                    southeast = MazeFlags.PassageNW;
                }
                else
                {
                    northwest = MazeFlags.PassageNE;
                    northeast = MazeFlags.PassageNW;
                    southeast = MazeFlags.PassageSW;
                }
            }
            else if (flipNS)
            {
                northwest = MazeFlags.PassageSW;
                northeast = MazeFlags.PassageSE;
                southeast = MazeFlags.PassageNE;
            }
            else
            {
                northwest = MazeFlags.PassageNW;
                northeast = MazeFlags.PassageNE;
                southeast = MazeFlags.PassageSE;
            }
        }
    }
}
```

Then add a static array containing the northeast, southeast, southwest, and northwest quadrants. This will be static data hard-coded into the job.

```
partial struct OcclusionJob
{
    …

    readonly static Quadrant[] quadrants = {
        new(false, false), // NE
        new(true, false), // SE
        new(true, true), // SW
        new(false, true), // NW
    };
}
```

2.2 Northeast Quadrant

Change `CellData.UpdateForNextCell` so it works for a given quadrant.

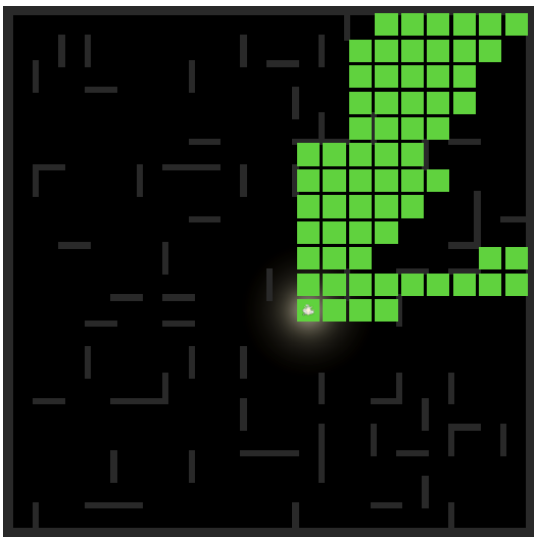
```
	public void UpdateForNextCell(MazeFlags cell, Quadrant quadrant) {
		northEastCornerSlope = east / north;

		if (cell.Has(quadrant.south) &&
			cell.HasNot(quadrant.southeast) &&
			SouthInset > 0f)
		{
			RightSlope = min(RightSlope, EastInset / SouthInset);
		}

		if (cell.Has(quadrant.north))
		{
			if (cell.HasNot(quadrant.northwest) && WestInset > 0f)
			{
				LeftSlope = max(LeftSlope, WestInset / NorthInset);
			}

			RightSlopeForNorthNeighbor = min(RightSlope,
				cell.Has(quadrant.northeast) ?
					northEastCornerSlope : EastInset / north);

			IsNorthVisible = LeftSlope < RightSlopeForNorthNeighbor;
		}
		else
		{
			RightSlopeForNorthNeighbor = 0f;
			IsNorthVisible = false;
		}

		IsEastVisible =
			cell.Has(quadrant.east) &&
			LeftSlope < RightSlope &&
			(cell.Has(quadrant.northeast) ?
				northEastCornerSlope < RightSlope :
				NorthInset > 0f && east / NorthInset < RightSlope);
	}
```

The scan, starting with the first, will depend on the quadrant, so add an output parameter for the quadrant to `OcclusionJob.TryGetFirstScan`, which simply returns the northeast quadrant.

```
	bool TryGetFirstScan(out Scan scan, out Quadrant quadrant, out float2 origin)
	{
		quadrant = quadrants[0];
		…
	}
```

Then adjust `Execute` to use the quadrant as well. The step directions are now also variable, based on which directions are flipped.

```
public void Execute(int i)
{
    if (!TryGetFirstScan(out Scan scan, out Quadrant quadrant, out float2 origin))
    {
        return;
    }

    int stepEast = quadrant.flipEW ? maze.StepW : maze.StepE;
    int stepNorth = quadrant.flipNS ? maze.StepS : maze.StepN;

    var stack = new ScanStack(maze.SizeEW, scan);
    while (stack.TryPop(out scan))
    {
        …

        while (true)
        {
            MazeFlags cell = maze.Set(currentIndex, MazeFlags.Visible);
            data.UpdateForNextCell(cell, quadrant);

            if (data.IsNorthVisible)
            {
                if (!isPreviousNorthVisible)
                {
                    scan.ShiftEast(currentIndex, currentXOffset, data.LeftSlope);
                }
                else if (cell.HasNot(quadrant.northwest))
                {
                    stack.Push(scan.ShiftedNorth(
                        stepNorth, previousRightSlopeForNorthNeighbor));
                    scan.ShiftEast(currentIndex, currentXOffset, data.LeftSlope);
                }
            }
            else if (isPreviousNorthVisible)
            {
                stack.Push(scan.ShiftedNorth(
                    stepNorth, previousRightSlopeForNorthNeighbor));
            }

            if (data.IsEastVisible)
            {
                currentIndex += stepEast;
                …
            }
            else
            {
                if (data.IsNorthVisible)
                {
                    stack.Push(scan.ShiftedNorth(
                        stepNorth, data.RightSlopeForNorthNeighbor));
                }
                break;
            }
        }
    }
}
```

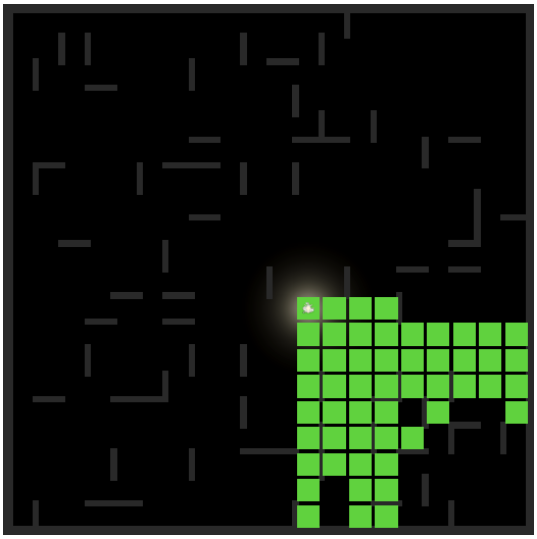


Northeast quadrant; player position (3, 0, −3).

2.3 Southeast Quadrant

Now switch to the southeast quadrant, by incrementing the index used in `TryGetFirstScan`. This flips the north-south direction of the algorithm. We also have to flip the origin's relative north-south position inside the cell to match and have to make sure that it stays less than 1.

```
        quadrant = quadrants[1];
        origin = frac(position);

        if (quadrant.flipNS)
        {
            origin.y = min(1f - origin.y, 0.999999f);
        }
```



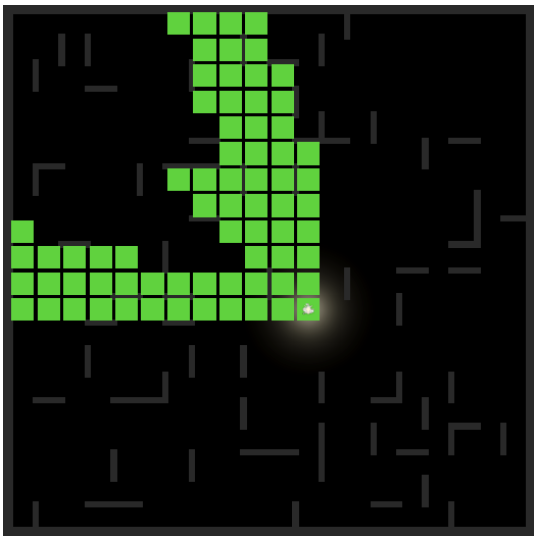
Southeast quadrant.

At this point we should see correct visibility data for the southeast quadrant. The algorithm doesn't care that we flipped one dimension.

2.4 Northwest Quadrant

Next we skip to the final quadrant, which is northwest, for which we have to flip the origin's relative east-west position.

```
        quadrant = quadrants[3];
        origin = frac(position);

        if (quadrant.flipEW)
        {
            origin.x = min(1f - origin.x, 0.999999f);
        }
```

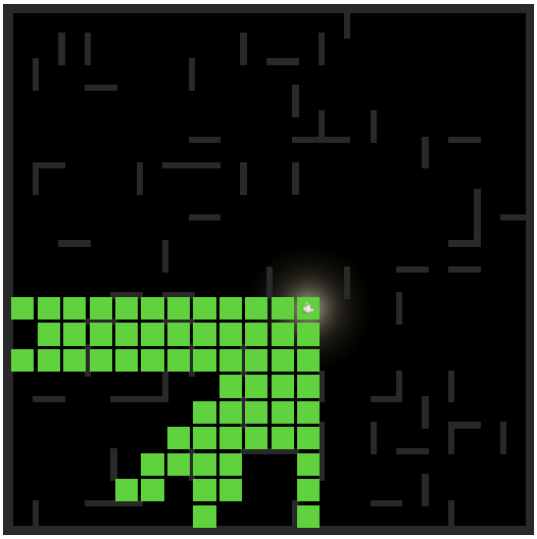


Northwest quadrant.

2.5 Southwest Quadrant

The last quadrant to check is southwest, which flips both dimensions.

```
        quadrant = quadrants[2];
```



Southwest quadrant.

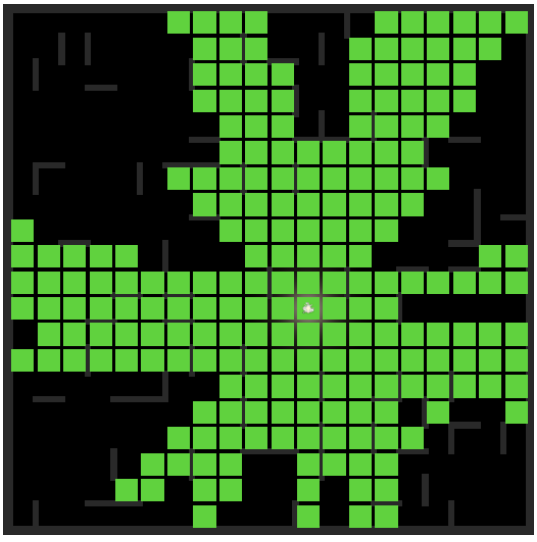
2.6 All Quadrants

To support all quadrants pass the job index from `Execute` to `TryGetFirstScan` and use it to retrieve a quadrant.

```
	public void Execute(int i)
	{
		if (!TryGetFirstScan(i, out Scan scan, out Quadrant quadrant, out float2 origin))
		{
			return;
		}

		…
	}

	bool TryGetFirstScan(int i, out Scan scan, out Quadrant quadrant, out float2 origin)
	{
		quadrant = quadrants[i];
		…
	}
```

Then increase the job length to four in `Game.UpdateOcclusion`.

```
		handle = new OcclusionJob
		{
			maze = maze,
			position = maze.WorldToMazePosition(playerPosition),
			wallExtents = maze.WorldToMazeDistance(wallExtents)
		}.ScheduleParallel(4, 1, handle);
```

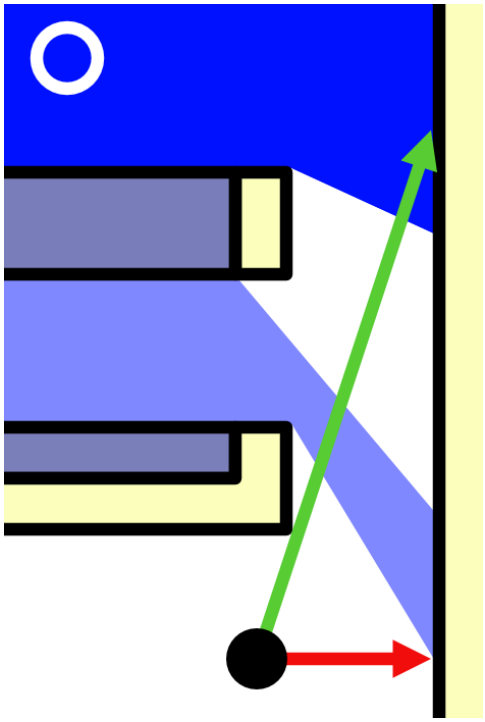


All quadrants.

Our occlusion algorithm now generates complete vision data for the player.

3 Agents

We can also determine visibility for our agents. This is needed because they contain shadow-casting lights. Cells that aren't visible to the player but are visible to an agent might cast shadows for light that the player could otherwise see.



Blue light leaking due to hiding cells invisible to player.

3.1 Visibility Flags

Add visibility flags to `MazeFlags` for our agents, along with masks for what's visible to them and what's visible to everyone. We support up to three agents, but more could be added. Also rename the generic `Visible` to the more specific `VisibleToPlayer`.

```
    VisibleToPlayer = 0b0001_0000_0000,

    VisbleToAgentA = 0b0010_0000_0000,
    VisbleToAgentB = 0b0100_0000_0000,
    VisbleToAgentC = 0b1000_0000_0000,

    VisibleToAllAgents = 0b1110_0000_0000,
    VisibleToAll = 0b1111_0000_0000
```

`ClearOcclusionJob` now has to clear all visibility flags.

```
    public void Execute(int i) => maze.Unset(i, MazeFlags.VisibleToAll);
```

And `OcclusionJob` needs a configuration field for the visibility flag that it has to set.

```
    public MazeFlags visibilityFlag;

    public void Execute(int i)
    {
        …
                MazeFlags cell = maze.Set(currentIndex, visibilityFlag);
        …
    }
```

Pass the player's visibility flag to the job in `Game.UpdateOcclusion`.

```
        handle = new OcclusionJob
        {
            maze = maze,
            position = maze.WorldToMazePosition(playerPosition),
            wallExtents = maze.WorldToMazeDistance(wallExtents),
            visibilityFlag = MazeFlags.VisibleToPlayer
        }.ScheduleParallel(4, 1, handle);
```

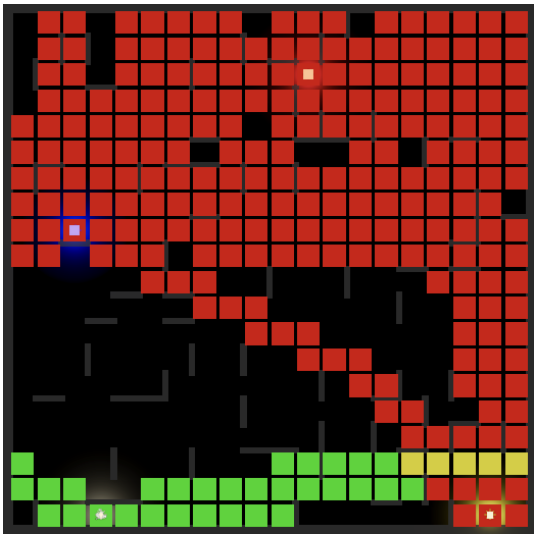
3.2 Occlusion for Agents

To also calculate occlusion for agents schedule jobs for them, after the player. We have to run them sequentially because they set different visibility flags, though each still processes its quadrants in parallel.

```
		float wallExtentsInMazeSpace = maze.WorldToMazeDistance(wallExtents);
		handle = new OcclusionJob
		{
			maze = maze,
			position = maze.WorldToMazePosition(playerPosition),
			wallExtents = wallExtentsInMazeSpace,
			visibilityFlag = MazeFlags.VisibleToPlayer
		}.ScheduleParallel(4, 1, handle);

		for (int i = 0; i < agents.Length; i++)
		{
			handle = new OcclusionJob
			{
				maze = maze,
				position = maze.WorldToMazePosition(agents[i].transform.localPosition),
				wallExtents = wallExtentsInMazeSpace,
				visibilityFlag = (MazeFlags)((int)MazeFlags.VisbleToAgentA << i)
			}.ScheduleParallel(4, 1, handle);
		}
```

Adjust `OnDrawGizmosSelected` so it shows all visible cells, using green for those visible to the player, red for those visible to agents, and yellow for those visible to both the player and an agent. Then add the red, blue, and yellow agents to the *Agents* array of *Game* again.

```
		//Gizmos.color = Color.green;
		var size = new Vector3(1.75f, 0.01f, 1.75f);
		for (int i = 0; i < cellObjects.Length; i++)
		{
			MazeFlags flags = maze[i];
			if (flags.HasAny(MazeFlags.VisibleToAll))
			{
				Gizmos.color = flags.Has(MazeFlags.VisibleToPlayer) ?
					flags.HasAny(MazeFlags.VisibleToAllAgents) ?
						Color.yellow : Color.green :
					Color.red;
				…
			}
		}
```



Both player and agent visibility.

3.3 Overlapping Visibility

We only care about light shed by agents that the player could potentially see. This is the case when the set of visible cells of the player and an agent overlap. We can detect this in `OcclusionJob` by keeping track of the flags that we spotted while running the algorithm. As the player goes first, if an agent spots the `VisibleToPlayer` flag then its light is visible to the player. We store that information in a native `bool` array with one element per agent. We also need a configuration field for the index to use. Then set the appropriate element to `true` at the end of the job, if needed and only for agents.

```
	[NativeDisableParallelForRestriction]
	public NativeArray<bool> isVisibleToPlayer;

	public int isVisibleToPlayerIndex;

	…

	public void Execute(int i)
	{
		…
		var spottedFlags = MazeFlags.Empty;

		var stack = new ScanStack(maze.SizeEW, scan);
		while (stack.TryPop(out scan))
		{
			…
			while (true)
			{
				MazeFlags cell = maze.Set(currentIndex, visibilityFlag);
				data.UpdateForNextCell(cell, quadrant);
				spottedFlags |= cell;

				…
			}
		}

		if (visibilityFlag != MazeFlags.VisibleToPlayer &&
			spottedFlags.Has(MazeFlags.VisibleToPlayer))
		{
			isVisibleToPlayer[isVisibleToPlayerIndex] = true;
		}
	}
```

Create a temporary array for the jobs, initialized to `false` in `Game.UpdateOcclusion`. We also have to pass the native array to the player's job even through it doesn't use it, otherwise the job system will complain.

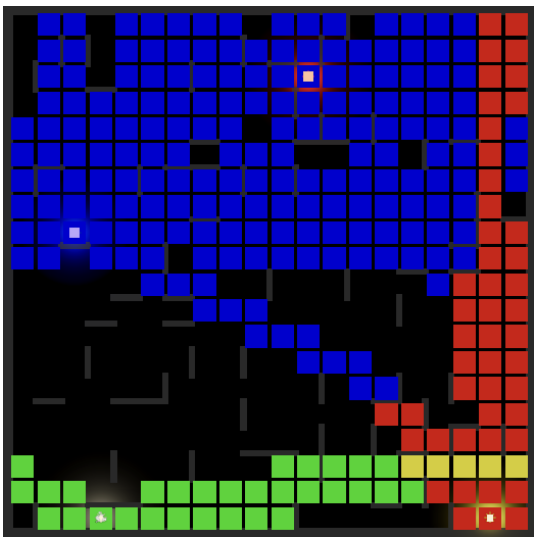
```
	void UpdateOcclusion(Vector3 playerPosition)
	{
		var isVisibleToPlayer = new NativeArray<bool>(agents.Length, Allocator.TempJob);
		JobHandle handle = new ClearOcclusionJob
		{
			maze = maze
		}.ScheduleParallel(maze.Length, maze.Length / 4, default);

		float wallExtentsInMazeSpace = maze.WorldToMazeDistance(wallExtents);
		handle = new OcclusionJob
		{
			isVisibleToPlayer = isVisibleToPlayer,
			…
		}.ScheduleParallel(4, 1, handle);

		for (int i = 0; i < agents.Length; i++)
		{
			handle = new OcclusionJob
			{
				isVisibleToPlayer = isVisibleToPlayer,
				isVisibleToPlayerIndex = i,
				…
			}.ScheduleParallel(4, 1, handle);
		}

		handle.Complete();

		isVisibleToPlayer.Dispose();
	}
```

Then we can use the array to set a visibility mask, for the player and all agents visible to the player. Make it a field so we can also access it in `OnDrawGizmosSelected`.

```
	MazeFlags visibilityMask;

	…

	void UpdateOcclusion(Vector3 playerPosition)
	{
		…

		visibilityMask = MazeFlags.VisibleToPlayer;
		for (int i = 0; i < agents.Length; i++)
		{
			if (isVisibleToPlayer[i])
			{
				visibilityMask |= (MazeFlags)((int)MazeFlags.VisbleToAgentA << i);
			}
		}
		isVisibleToPlayer.Dispose();
	}
```

Use the visibility mask to give cells that are safe to ignore a blue color.

```
				Gizmos.color = flags.Has(MazeFlags.VisibleToPlayer) ?
					flags.HasAny(MazeFlags.VisibleToAllAgents) ?
						Color.yellow : Color.green :
					flags.HasAny(visibilityMask) ?
						Color.red : Color.blue;
```



Blue cells can also be hidden.

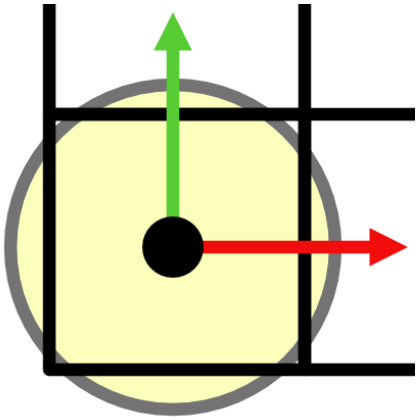
4 Finalizing

At this point we have a fully-functional algorithm, but it can be improved a bit further before we use it to hide cells.

4.1 Range

The lights of the agents have limited range, which we currently ignore. Let's support a maximum vision range as this can drastically reduce the amount of visible cells.

Add a range parameter to `CellData.UpdateForNextCell` and use that to check whether the north and east neighbors are in range, which can be done via a squared-distance comparison. North can only be visible if the northwest corner is in range. However, this might fail if west lies outside the quadrant. So clamp west to zero for this check. East can only be visible if the southeast corner is in range, with south clamped for the same reason.



Range checks could fail without clamping.

```
	public void UpdateForNextCell(MazeFlags cell, Quadrant quadrant, float range) {
		…

		if (cell.Has(quadrant.north) && IsInRange(max(0f, west), north, range))
		{
			…
		}
		else
		{
			RightSlopeForNorthNeighbor = 0f;
			IsNorthVisible = false;
		}

		IsEastVisible =
			cell.Has(quadrant.east) &&
			LeftSlope < RightSlope &&
			IsInRange(east, max(0f, south), range) &&
			(cell.Has(quadrant.northeast) ?
				northEastCornerSlope < RightSlope :
				NorthInset > 0f && east / NorthInset < RightSlope);
	}

	static bool IsInRange(float x, float y, float range) =>
		(x * x + y * y) < range * range;
```

Add a configuration field for the range to `OcclusionJob` and use it when updating the cell data.

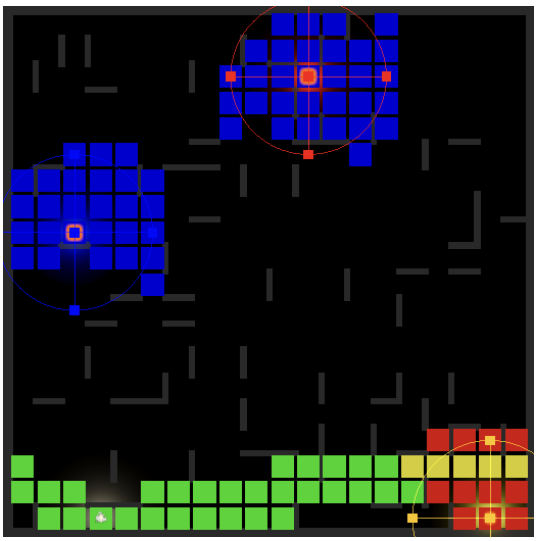
```
	public float range;

	…

	public void Execute(int i)
	{
		…
			data.UpdateForNextCell(cell, quadrant, range);
		…
	}
```

Make `Agent` keep track of its point light and expose its range via a property.

```
	Light pointLight;

	public float LightRange => pointLight.range;

	public string TriggerMessage => triggerMessage;

	void Awake()
	{
		pointLight = GetComponent<Light>();
		pointLight.color = color;
		…
	}
```

Then pass the ranges to the occlusion jobs in `Game.UpdateOcclusion`, converted to maze space. We also have to provide a range for the player, for which we'll use 1000.

```
        handle = new OcclusionJob
        {
            …
            position = maze.WorldToMazePosition(playerPosition),
            range = 1000f,
            …
        }.ScheduleParallel(4, 1, handle);

        for (int i = 0; i < agents.Length; i++)
        {
            Agent agent = agents[i];
            handle = new OcclusionJob
            {
                …
                position = maze.WorldToMazePosition(agent.transform.localPosition),
                range = maze.WorldToMazeDistance(agent.LightRange),
                …
            }.ScheduleParallel(4, 1, handle);
        }
```



Limited light range.

Could we also limit the view range of the player?

Yes, but that requires special care so agents and their lights and trails don't suddenly pop in and out of view. It can be done by adding fully-blocking fog to the maze. That's a potential feature to add in the future.

4.2 Field of View

The player has a limited field of view cone, that we currently ignore. Let's add support for it. Begin by introducing a convenient `FieldOfView` struct that combines 2D left and right sight lines with a view range. Also add an indication whether the field of view is omnidirectional, in which case the sight lines will be ignored.

```
using Unity.Mathematics;

public struct FieldOfView
{
	public float2 leftLine, rightLine;

	public float range;

	public bool omnidirectional;
}
```

Add a field for the field of view to `Player` with a range of 1000, along with a property to get it. Also make it keep track of its eye camera.

```
	Camera eyeCamera;

	FieldOfView vision;

	public FieldOfView Vision => vision;

	void Awake()
	{
		characterController = GetComponent<CharacterController>();
		eye = transform.GetChild(0);
		eyeCamera = eye.GetComponent<Camera>();
		vision.range = 1000f;
	}
```

We set the view lines in `UpdateEyeAngles`. Use the eye camera to find the X and Y view factors. Use the X factor to create left and right lines in the XZ plane and apply the eye rotation to them, then use that to set the vision lines.

```
	void UpdateEyeAngles()
	{
		…
		var rotation = eye.localRotation = Quaternion.Euler(eyeAngles.y, eyeAngles.x, 0f);

		float viewFactorY = Mathf.Tan(eyeCamera.fieldOfView * 0.5f * Mathf.Deg2Rad);
		float viewFactorX = viewFactorY * eyeCamera.aspect;

		Vector3
			leftLine = rotation * new Vector3(-viewFactorX, 0f, 1f),
			rightLine = rotation * new Vector3(viewFactorX, 0f, 1f);
		vision.leftLine = float2(leftLine.x, leftLine.z);
		vision.rightLine = float2(rightLine.x, rightLine.z);
	}
```

How did you find those view factors?

See Runner 2 / Skyline / Filling the View.

This would work if we always look straight ahead, but when looking up or down the vision cone widens when projected on the XZ plane. We can take that into account by making the lines point up or down before rotating, based on the Y factor. If the vertical rotation is negative then we're looking up and should use the top lines of the view frustum, otherwise the bottom lines.

```
		float y = eyeAngles.y < 0f ? viewFactorY : -viewFactorY;
		Vector3
			leftLine = rotation * new Vector3(-viewFactorX, y, 1f),
			rightLine = rotation * new Vector3(viewFactorX, y, 1f);
```

Next, replace the range configuration field of `OcclusionJob` with one for the field of view.

```
	//public float range;
	public FieldOfView fieldOfView;

	public MazeFlags visibilityFlag;

	public void Execute(int i)
	{
		…
				data.UpdateForNextCell(cell, quadrant, fieldOfView.range);
		…
	}
```

Provide the player's field of view in `Game.UpdateOcclusion` and create omnidirectional ones for the agents.

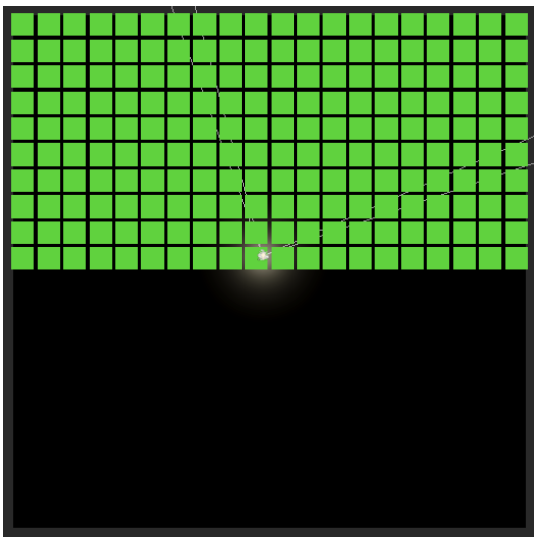
```
		handle = new OcclusionJob
		{
			…
			//range = 1000f,
			fieldOfView = player.Vision,
			…
		}.ScheduleParallel(4, 1, handle);

		for (int i = 0; i < agents.Length; i++)
		{
			Agent agent = agents[i];
			handle = new OcclusionJob
			{
				…
				fieldOfView = new FieldOfView
				{
					range = maze.WorldToMazeDistance(agent.LightRange),
					omnidirectional = true
				},
				…
			}.ScheduleParallel(4, 1, handle);
		}
```

Before we can apply the sight lines in `OcclusionJob.TryGetFirstScan` we must first adapt them to the quadrant we're working on. If east-west is flipped then negate their X coordinates. If north-south is flipped then negate their Y coordinates. Also, if there is only a single flip then we have to first swap the left and right lines.

```
float2 leftLine, rightLine;
if (quadrant.flipEW != quadrant.flipNS)
{
    leftLine = fieldOfView.rightLine;
    rightLine = fieldOfView.leftLine;
}
else
{
    leftLine = fieldOfView.leftLine;
    rightLine = fieldOfView.rightLine;
}

if (quadrant.flipEW)
{
    leftLine.x = -leftLine.x;
    rightLine.x = -rightLine.x;
    origin.x = min(1f - origin.x, 0.999999f);
}

if (quadrant.flipNS)
{
    leftLine.y = -leftLine.y;
    rightLine.y = -rightLine.y;
    origin.y = min(1f - origin.y, 0.999999f);
}
```

Unless the field of view is omnidirectional, we can potentially skip up to three quadrants. If the left line's X coordinate is at least zero while its Y coordinate is at most zero then we're looking to the right of the quadrant and can skip it. Otherwise, if right X is at most zero while its Y is at least zero then we're looking to the left of the quadrant and can skip it. Otherwise, if both left Y and right X are at most zero then we're looking in the opposite direction of the quadrant and can skip it. This works because angle between the lines is less than 180°.

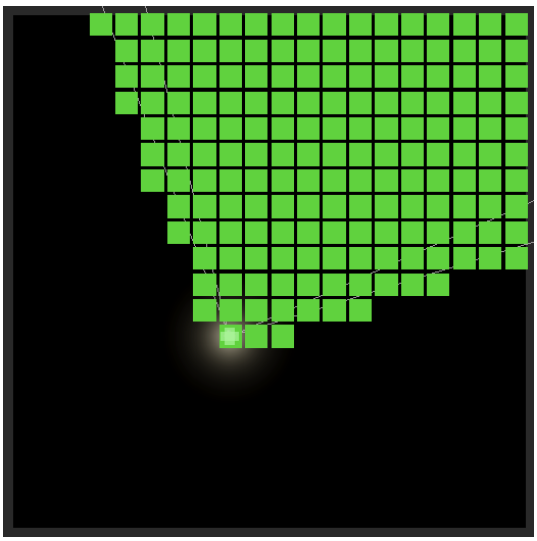
```
if (fieldOfView.omnidirectional)
{
    return true;
}

if (
    leftLine.x >= 0f && leftLine.y <= 0f ||
    rightLine.x <= 0f && rightLine.y >= 0f ||
    leftLine.y <= 0f && rightLine.x <= 0f)
{
    return false;
}

return true;
```



Skipped two quadrants; fully open maze.

If we do get a valid scan without omnidirectional vision then constrain the scan's slopes based on the lines.

```
		scan.leftSlope = leftLine.x <= 0f ? 0f : leftLine.x / leftLine.y;
		scan.rightSlope = rightLine.y <= 0f ? float.MaxValue : rightLine.x / rightLine.y;
		return true;
```



Applied field of view; fully open maze.

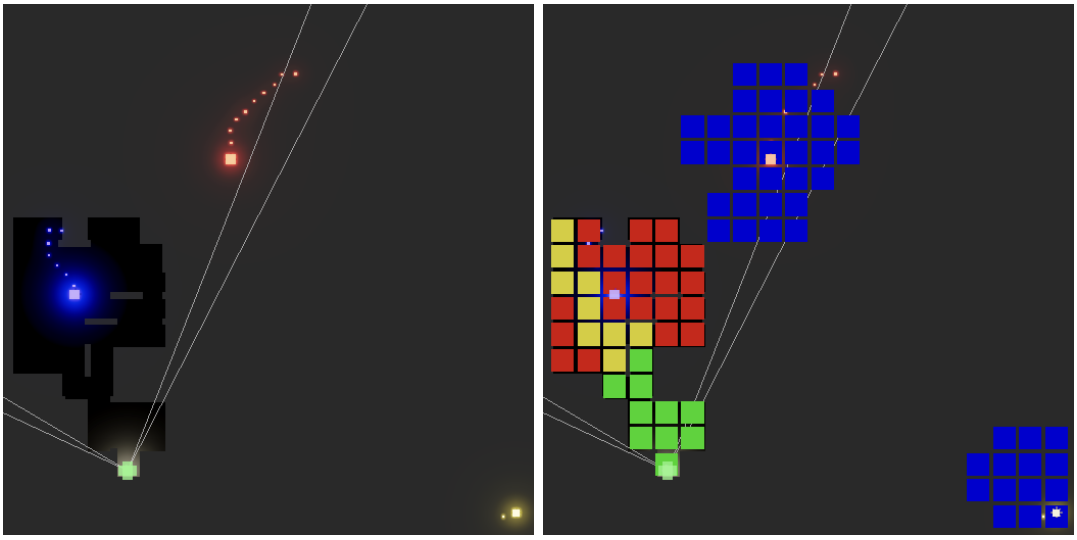
Could we use this for spotlights as well?

Yes. For maximum benefit you should also adjust the light range depending on how much the light points up or down. The same is true for the player if we would support limited sight range for it.

4.3 Hiding Cells and Lights

To finally take advantage of our occlusion system, activate or deactivate all cell objects based on the visibility mask, at the end of `Game.UpdateOcclusion`.

```
		isVisibleToPlayer.Dispose();

		for (int i = 0; i < cellObjects.Length; i++)
		{
			cellObjects[i].gameObject.SetActive(maze[i].HasAny(visibilityMask));
		}
```



Hiding cells; without and with visibility gizmos.

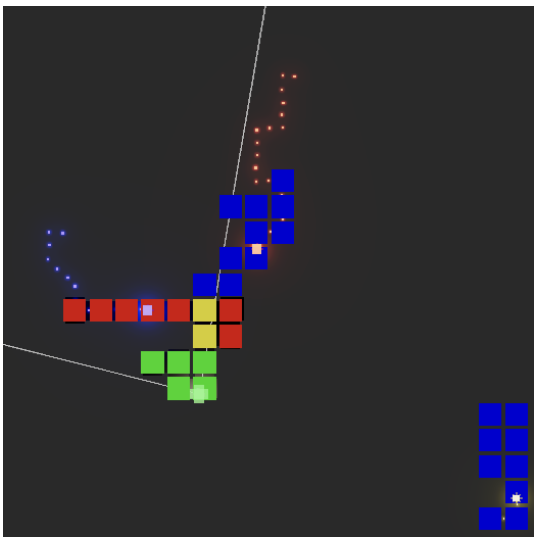
We should also turn off the lights of agents that are completely hidden. Add a `SetLightEnabled` method for that to `Agent`.

```
	public void SetLightEnabled (bool enabled) => pointLight.enabled = enabled;
```

Then enable or disable their lights in `Game.UpdateOcclusion`, which can be done while we construct the visibility mask.

```
		for (int i = 0; i < agents.Length; i++)
		{
			bool isVisible = isVisibleToPlayer[i];
			agents[i].SetLightEnabled(isVisible);
			if (isVisible)
			{
				visibilityMask |= (MazeFlags)((int)MazeFlags.VisbleToAgentA << i);
			}
		}
```

Our occlusion system is now fully operational. It won't provide much benefit for wide-open mazes, but it can drastically reduce the amount of work needed to render a typical maze with winding passages.



Only light from one agent visible; open arbitrary probability 0.25.

The next tutorial is Maze 2.2.0.

made by Jasper Flick